

Segmentação de Imagens de Microtomografia com KAN

CPE883 - Tópicos Especiais em Machine Learning
Vivian Rodrigues (DRE:125228569)

1. Introdução ao Problema

- Segmentação automática de imagens de microtomografia de rochas.
- Objetivo: Identificar regiões de fundo, rocha e poro em imagens 2D.
- Motivação: Suporte à análise de porosidade e estrutura interna de materiais.

2. Dataset

- 4 amostras de micro tomografia
- Total de imagens de entrada: 5558 (8.5 Gb em disco)
- Total de máscaras: 5558 (4.3 Gb em disco)
- Resolução das imagens de entrada 40 microns

3. Máscaras Multiescala (pré-tratamento)

```
# Função para extrair o número do slice do nome do arquivo (ex: "rocha0001.tiff" -> "0001")
```

```
def extrair_numero(nome_arquivo):  
    m = re.search(r'\d+', nome_arquivo)  
    return m.group(0) if m else None
```

```
# Listar todos arquivos da pasta que começam com 'rocha' e terminam com '.tiff'
```

```
arquivos_rocha = [f for f in os.listdir(pasta) if f.startswith('rocha') and f.endswith('.tiff')]
```

```
print(f"{len(arquivos_rocha)} arquivos de rocha encontrados.")
```

```
for arquivo_rocha in sorted(arquivos_rocha):
```

```
    numero = extrair_numero(arquivo_rocha)
```

```
    if numero is None:
```

```
        print(f"Não consegui extrair número do arquivo {arquivo_rocha}, pulando...")
```

```
        continue
```

```
    arquivo_poros = f"poro{numero}.tiff"
```

```
    arquivo_fundo = f"fundo{numero}.tiff"
```

```
    saida = f"mask{numero}.tiff"
```

```
# Carregar imagens
```

```
rocha = iio.imread(caminho_rocha)
```

```
poros = iio.imread(caminho_poros)
```

```
fundo = iio.imread(caminho_fundo)
```

```
# Converter 255 -> 1
```

```
rocha_bin = (rocha > 128).astype(np.uint8)
```

```
poros_bin = (poros > 128).astype(np.uint8)
```

```
fundo_bin = (fundo > 128).astype(np.uint8)
```

```
# Criar máscara multiclasse
```

```
mask = np.zeros_like(rocha_bin, dtype=np.uint8)
```

```
mask[fundo_bin == 1] = 0
```

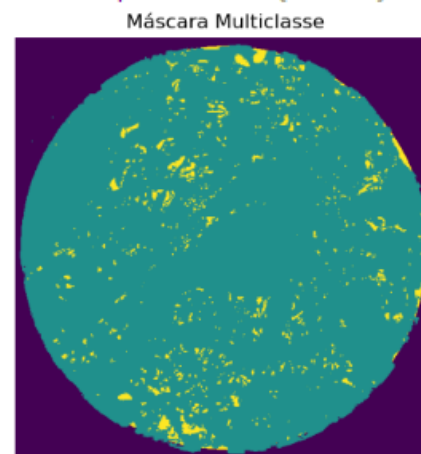
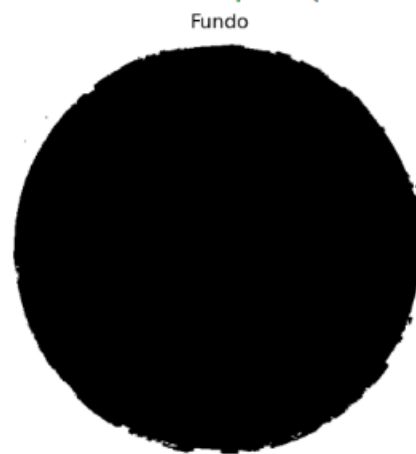
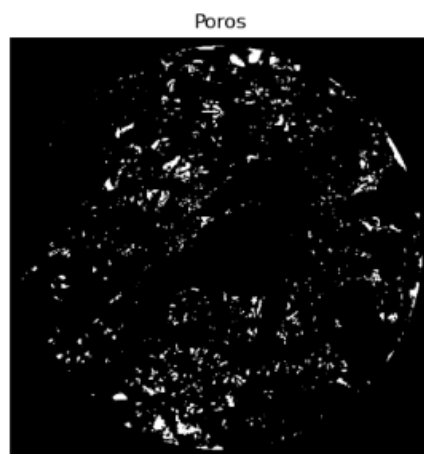
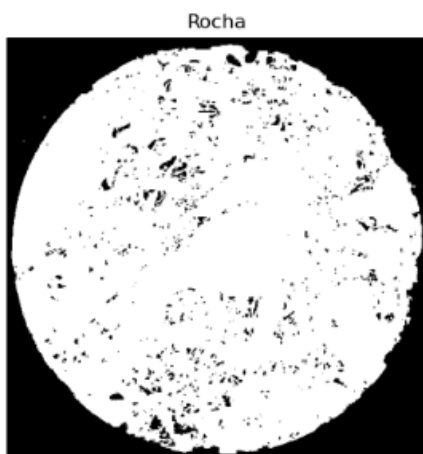
```
mask[rocha_bin == 1] = 1
```

```
mask[poros_bin == 1] = 2
```

```
# Salvar máscara
```

```
iio.imwrite(caminho_saida, mask)
```

```
print(f"Máscara salva para slice {numero} em {caminho_saida}")
```



4. Dataloader para Segmentação

- Este dataloader retorna imagens normalizadas e máscaras associadas, redimensionadas para um tamanho fixo
- Percorre todas as subpastas (DatasetA, B, ...), procurando por inputs/ e masks/.
- Leitura normalização da imagem + leitura de máscara
- Redimensionamento (512 x 512 pixels)
- Conversão para tensores

```
class RockSegmentationDatasetMulti(Dataset):  
    def __getitem__(self, idx):  
        image = iio.imread(input_path).astype(np.float32) / 255.0  
        mask = iio.imread(mask_path).astype(np.int64)  
        image = resize(image, self.output_shape)  
        mask = resize(mask, self.output_shape, order=0).astype(np.int64)  
        image = np.expand_dims(image, axis=0)  
        return torch.tensor(image), torch.tensor(mask), amostra_nome
```

python-repl

```
dataset/  
├─ DatasetA/  
│   ├─ inputs/  
│   └─ masks/  
├─ DatasetB/  
│   ├─ inputs/  
│   └─ masks/  
...
```

5. Experimento 1: Classificação global

- Dataloader com Classe Dominante
- Adaptação para tarefa de classificação: extrai a classe dominante da máscara (0: fundo, 1: rocha, 2: poro) como rótulo único.

```
def __getitem__(self, idx):  
    ...  
    unique, counts = np.unique(mask, return_counts=True)  
    dominante = int(unique[np.argmax(counts)])  
    return torch.tensor(image), torch.tensor(dominante), amostra_nome
```

5. Experimento 1: Classificação global

- Modelo rede convolucional (Classificação Global)
- Modelo simples com convoluções 1x1 e ativação ReLU para simular CKAN.
- A saída é um vetor de 3 classes da classe dominante.

```
class SimpleCKANModel(nn.Module):
    def __init__(self):
        ...

        self.ckan2 = CKANConv2D(16, 32)
        self.pool2 = nn.MaxPool2d(2)

        self.ckan3 = CKANConv2D(32, 64)
        self.pool3 = nn.AdaptiveAvgPool2d((8, 8)) # Reduz tamanho para FC

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 8 * 8, 128),
            nn.ReLU(),
            nn.Linear(128, 3),
        )
```

#[Entrada da imagem 512x512x1]

CKANConv2D(1→16)	(Conv2d 1x1 + ReLU)
MaxPool2D	(reduz para 256x256)
CKANConv2D(16→32)	(Conv2d 1x1 + ReLU)
MaxPool2D	(reduz para 128x128)
CKANConv2D(32→64)	(Conv2d 1x1 + ReLU)
AdaptiveAvgPool(8x8)	(reduz para 8x8, fixo)
Flatten	
FC (Linear: 64*8*8 → 128)	
ReLU	
FC (Linear: 128 → 3 classes)	

5. Experimento 1: Classificação global

- Otimização Adam
- Função objetiva entropia cruzada

```
@app.command()
def train(
    data_dir: str = typer.Option(r"..\\mini_dataset", help="Caminho para o dataset"),
    epochs: int = 10,
    batch_size: int = 4,
    lr: float = 1e-3
):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    dataset = RockSegmentationDatasetMulti(data_dir, output_shape=(128, 128)) # menor p/ treino rápido
    dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

    model = SimpleCKANModel(num_classes=3).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)
```


6. Segmentação com CKAN

- Camadas convolucionais padrão (Conv2D) combinadas com funções da ativação KAN (splines aprendíveis)
- A última camada gera um mapa de classes com mesma resolução da entrada.

```
class CKANSegmentationModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.ckan1 = CKANConv2D(1, 16)
        self.pool1 = nn.Identity() #nn.MaxPool2d(2)

        self.ckan2 = CKANConv2D(16, 32)
        self.pool2 = nn.Identity() #nn.MaxPool2d(2)

        self.ckan3 = CKANConv2D(32, 64)
        self.pool3 = nn.Identity() #nn.AdaptiveAvgPool2d((64, 64)) # shape final fixo

        self.final_conv = nn.Conv2d(64, num_classes, kernel_size=1) # saída: [B, 3, H, W]

    def forward(self, x):
        ...
        return x # [B, 3, H, W]
```

[Input: 128×128×1]

CKANConv2D(1 → 16) # Conv2D(1→16) + KANActivation(16)
Identity (sem pooling)

CKANConv2D(16 → 32) # Conv2D(16→32) + KANActivation(32)
Identity (sem pooling)

CKANConv2D(32 → 64) # Conv2D(32→64) + KANActivation(64)
Identity (sem pooling)

Conv2D(64 → 3) # Camada final para gerar mapa de classes

[Output: 128×128×3] # Segmentação por pixel (3 classes)

6. Segmentação com CKAN

- Ativação KAN
- Ativação personalizada baseada aplica splines 1D aprendíveis individualmente em cada canal da magem

```
class KANActivation(nn.Module):
    def __init__(self, num_channels):
        super().__init__()
        # Cria uma KAN independente para cada canal
        self.kans = nn.ModuleList([KAN(width=[1, 1]) for _ in range(num_channels)])
        self.num_channels = num_channels

    def forward(self, x):
        # x: [B, C, H, W]
        B, C, H, W = x.shape
        x_split = torch.split(x, 1, dim=1)          # lista de [B,1,H,W] para cada canal

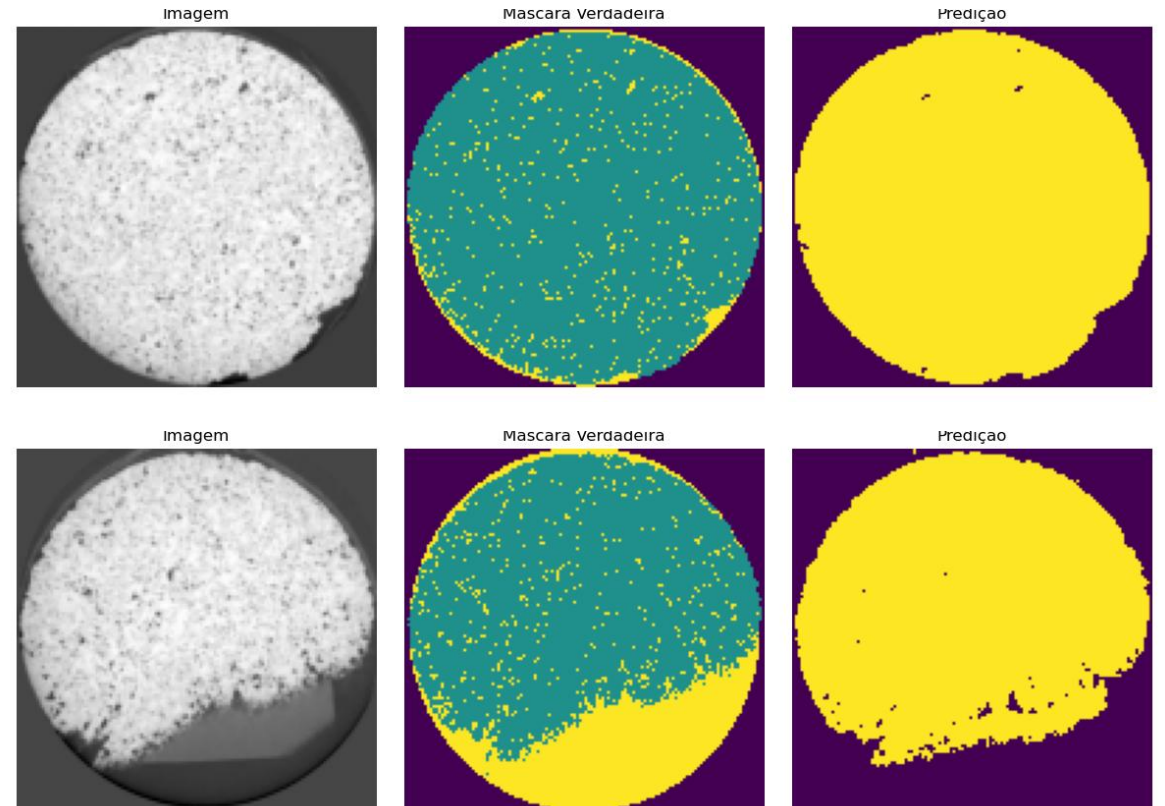
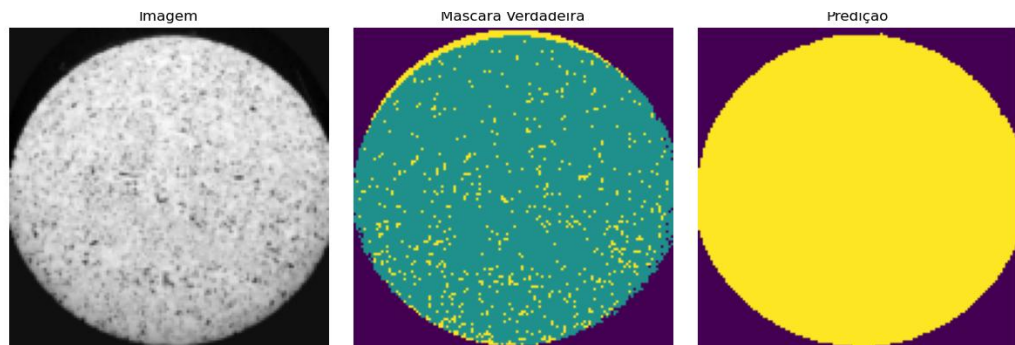
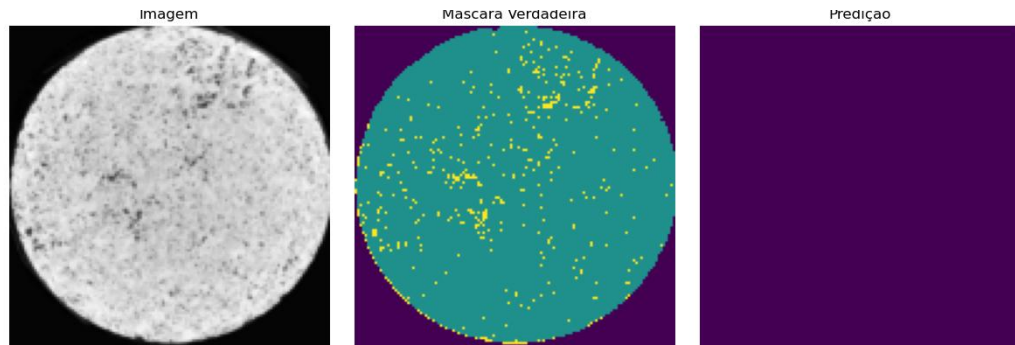
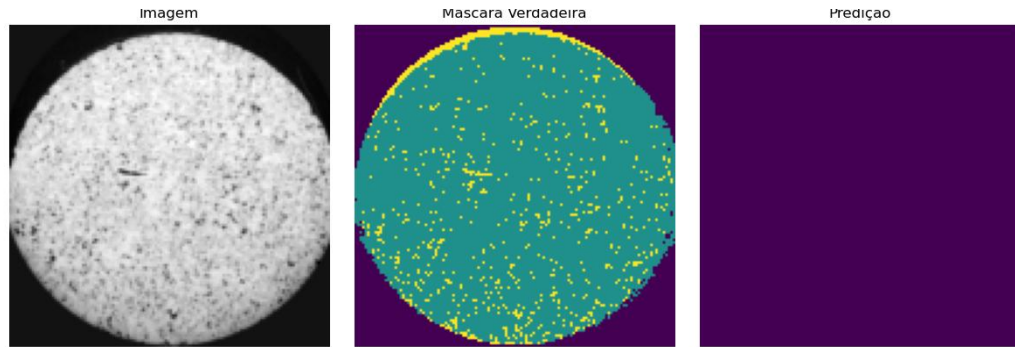
        out_channels = []
        for i, xi in enumerate(x_split):
            xi_flat = xi.reshape(B * H * W, 1)      # corrigido: .reshape() ao invés de .view()
            yi_flat = self.kans[i](xi_flat)          # aplica KAN individual
            yi = yi_flat.view(B, 1, H, W)           # esta view ainda é válida, tensor novo
            out_channels.append(yi)

        out = torch.cat(out_channels, dim=1)        # [B, C, H, W]
        return out
```

7. Segmentação com CKAN

- ✓ Treinamento funcional em CPU
- ✓ Pipeline completo com segmentação por pixel
- ✓ Visualização qualitativa a cada época (imagem, máscara, predição)
- ✓ Acurácia preliminar reportada por época
- ✓ Resultados qualitativos são exibidos por época, incluindo imagem original, máscara verdadeira e predição do modelo.

7. Resultados Preliminares



8. Próximos Passos

- **Plotar curva de erro (loss)** ao longo do treinamento
- Verificar suporte a **KAN com splines 2D** (aplicação espacial) – Pykan 0.2.8
- Substituir filtros convolucionais 1×1 por **filtros 3×3**
- Implementar métricas de validação
- Desafios
 - Introduzir arquitetura **Encoder/Decoder** (como U-Net)
 - Criar uma versão adaptada da **U-Net com CKAN**

Referências

- Paper original: "Kolmogorov–Arnold Networks" (Ziming Liu et al., 2024)
- Apresentação Seminário KAN